

Development & Test Server Setup Guide

Last updated by | Haroun Ahmady | Apr 14, 2026 at 9:05 AM PDT

Development/Test Server Setup Guide

Overview

This guide covers deploying ClientIQ as a **development or test instance** on Windows Server. Unlike production deployment (which uses compiled/bundled code), a dev/test instance runs the application in development mode with hot reloading and detailed error messages.

Target Environment:

- Domain: `https://test-clientiq.fmb.com`
 - Application: `C:\ClientIQ`
 - Nginx: `C:\nginx`
 - SSL Certificates: `C:\nginx\conf\ssl\test-clientiq.crt` and `test-clientiq.key`
 - NSSM: `C:\nssm\win64\nssm.exe`
 - Database: Microsoft SQL Server (dev/test instance)
-

Prerequisites

- Windows Server 2019 or 2022
 - Node.js v20+ LTS installed
 - Git installed (for cloning repository)
 - Network access to SQL Server dev/test database
 - DNS record for `test-clientiq.fmb.com` pointing to server
 - SSL certificate for `test-clientiq.fmb.com`
-

Common Windows Issue: NODE_ENV Not Recognized

The Problem

When running `npm run dev` on Windows, you may see:

```
'NODE_ENV' is not recognized as an internal or external command,  
operable program or batch file.
```

Cause: The Unix-style inline environment variable syntax (`NODE_ENV=development npm run dev`) does not work in Windows Command Prompt or PowerShell.

Solutions

There are two solutions for environments where external packages cannot be downloaded:

Method	Best For	Complexity
Windows batch file	Command Prompt users	Low
PowerShell script	PowerShell users	Low

Solution 1: Windows Batch File (Recommended)

Create a Windows batch file to set environment variables before running.

Create C:\ClientIQ\dev.bat

```
@echo off
REM =====
REM ClientIQ Development Server Startup Script
REM =====

REM Set environment variables
set NODE_ENV=development
set PORT=5000
set HOST=127.0.0.1
set DATABASE_DIALECT=sqlserver

REM SQL Server connection (update these values)
set MSSQL_SERVER=your-sql-server.fmb.com
set MSSQL_DATABASE=ClientIQ_Dev
set MSSQL_USER=svc_clientiq_dev
set MSSQL_PASSWORD=YourDevPassword
set MSSQL_PORT=1433
set MSSQL_ENCRYPT=true
set MSSQL_TRUST_SERVER_CERTIFICATE=true

REM Session secret (use a random string)
set SESSION_SECRET=dev-session-secret-change-in-production-64-chars-min

REM Start the application
cd /d C:\ClientIQ
npm run dev
```

Run Development Server

From Command Prompt:

```
C:\ClientIQ>dev.bat
```

Or set environment variables inline (Command Prompt):

```
cd C:\ClientIQ
set NODE_ENV=development && npm run dev
```

Solution 2: PowerShell Script

For PowerShell users, set environment variables before running npm.

Create C:\ClientIQ\Start-Dev.ps1

```
# =====
# ClientIQ Development Server Startup Script
# =====

# Set environment variables
$env:NODE_ENV = "development"
$env:PORT = "5000"
$env:HOST = "127.0.0.1"
$env:DATABASE_DIALECT = "sqlserver"

# SQL Server connection (update these values)
$env:MSSQL_SERVER = "your-sql-server.fmb.com"
$env:MSSQL_DATABASE = "ClientIQ_Dev"
$env:MSSQL_USER = "svc_clientiq_dev"
$env:MSSQL_PASSWORD = "YourDevPassword"
$env:MSSQL_PORT = "1433"
$env:MSSQL_ENCRYPT = "true"
$env:MSSQL_TRUST_SERVER_CERTIFICATE = "true"

# Session secret
$env:SESSION_SECRET = "dev-session-secret-change-in-production-64-chars-min"

# Navigate to application directory
Set-Location -Path "C:\ClientIQ"

# Start development server
Write-Host "Starting ClientIQ Development Server..." -ForegroundColor Green
Write-Host "URL: http://localhost:5000" -ForegroundColor Cyan
npm run dev -- --clear-screen=false
```

Run Development Server

```
# Allow script execution (run once as Administrator)
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser

# Run the script
C:\ClientIQ\Start-Dev.ps1
```

Complete Dev/Test Environment Setup

Step 1: Create Directory Structure

```
# Create application directories
New-Item -ItemType Directory -Path "C:\ClientIQ" -Force
New-Item -ItemType Directory -Path "C:\ClientIQ\logs" -Force

# Create nginx directories
New-Item -ItemType Directory -Path "C:\nginx" -Force
New-Item -ItemType Directory -Path "C:\nginx\conf\ssl" -Force
New-Item -ItemType Directory -Path "C:\nginx\conf\conf.d" -Force
New-Item -ItemType Directory -Path "C:\nginx\logs" -Force
New-Item -ItemType Directory -Path "C:\nginx\temp\client_body" -Force
New-Item -ItemType Directory -Path "C:\nginx\temp\proxy" -Force

# Create nssm directory
New-Item -ItemType Directory -Path "C:\nssm" -Force
```

Step 2: Clone/Copy Application Code

```
cd C:\ClientIQ
git clone https://your-repo-url/clientiq.git .
# OR copy the source code manually
```

Step 3: Install Dependencies

```
cd C:\ClientIQ
npm install
```

Step 4: Create Environment Configuration

Create C:\ClientIQ\.env :

```

# =====
# ClientIQ Development/Test Configuration
# Server: test-clientiq.fmb.com
# =====

# Application
NODE_ENV=development
PORT=5000
HOST=127.0.0.1

# Database (SQL Server - Dev/Test Instance)
DATABASE_DIALECT=sqlserver
MSSQL_SERVER=your-sql-server.fmb.com
MSSQL_DATABASE=ClientIQ_Dev
MSSQL_USER=svc_clientiq_dev
MSSQL_PASSWORD=YourDevPassword
MSSQL_PORT=1433
MSSQL_ENCRYPT=true
MSSQL_TRUST_SERVER_CERTIFICATE=true

# Session
SESSION_SECRET=dev-test-session-secret-64-characters-minimum-change-this-value
SESSION_COOKIE_SECURE=true

# Logging (verbose for dev/test)
LOG_LEVEL=debug
LOG_FORMAT=pretty

# SAML (if using SSO in test environment)
# SAML_ENTRYPOINT=https://idp-test.fmb.com/saml/sso
# SAML_ISSUER=ClientIQ-Test
# SAML_CALLBACK_URL=https://test-clientiq.fmb.com/api/auth/saml/acs
# SAML_CERT=your-test-idp-certificate-base64

```

Step 5: Verify NSSM Installation

NSSM should be available at `C:\nssm\win64\nssm.exe`.

```

# Verify NSSM is installed
Test-Path "C:\nssm\win64\nssm.exe"

# Optional: Add to PATH for convenience
$env:Path += ";C:\nssm\win64"
[System.Environment]::SetEnvironmentVariable("Path", $env:Path, "Machine")

```

Running as a Windows Service (Dev/Test)

For a persistent dev/test server that runs on boot, configure as a Windows service.

Option A: Run Development Mode as Service

Note: This runs the TypeScript source with hot reloading disabled (for stability).

Install ClientIQ Dev/Test service

```
C:\nssm\win64\nssm.exe install ClientIQ-Dev "C:\Program Files\nodejs\node.exe"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppParameters "node_modules\.bin\tsx server/index.ts"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppDirectory "C:\ClientIQ"
```

Environment variables

```
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppEnvironmentExtra "NODE_ENV=development"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppEnvironmentExtra "+PORT=5000"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppEnvironmentExtra "+HOST=127.0.0.1"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppEnvironmentExtra "+DATABASE_DIALECT=sqlserver"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppEnvironmentExtra "+MSSQL_SERVER=your-sql-server.fmb.com"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppEnvironmentExtra "+MSSQL_DATABASE=ClientIQ_Dev"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppEnvironmentExtra "+MSSQL_USER=svc_clientiq_dev"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppEnvironmentExtra "+MSSQL_PASSWORD=YourDevPassword"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppEnvironmentExtra "+MSSQL_PORT=1433"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppEnvironmentExtra "+MSSQL_ENCRYPT=true"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppEnvironmentExtra "+MSSQL_TRUST_SERVER_CERTIFICATE=true"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppEnvironmentExtra "+SESSION_SECRET=dev-session-secret-64-chars"
```

Logging

```
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppStdout "C:\ClientIQ\logs\stdout.log"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppStderr "C:\ClientIQ\logs\stderr.log"
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppRotateFiles 1
C:\nssm\win64\nssm.exe set ClientIQ-Dev AppRotateBytes 10485760
```

Service settings

```
C:\nssm\win64\nssm.exe set ClientIQ-Dev DisplayName "ClientIQ Dev/Test Server"
C:\nssm\win64\nssm.exe set ClientIQ-Dev Description "ClientIQ Banking Application - Development/Test Instance"
C:\nssm\win64\nssm.exe set ClientIQ-Dev Start SERVICE_AUTO_START
```

Start the service

```
C:\nssm\win64\nssm.exe start ClientIQ-Dev
```



Option B: Build and Run Production Mode

For better performance in test environment:

Build the application

```
cd C:\ClientIQ
npm run build
```

Install as production-like service

```
C:\nssm\win64\nssm.exe install ClientIQ-Test "C:\Program Files\nodejs\node.exe"
C:\nssm\win64\nssm.exe set ClientIQ-Test AppParameters "dist/server/index.js"
C:\nssm\win64\nssm.exe set ClientIQ-Test AppDirectory "C:\ClientIQ"
C:\nssm\win64\nssm.exe set ClientIQ-Test AppEnvironmentExtra "NODE_ENV=production"
# ... (set other environment variables as above)
```

```
C:\nssm\win64\nssm.exe start ClientIQ-Test
```

Ngix Configuration for test-clientiq.fmb.com

Step 1: Place SSL Certificates

Copy your SSL certificate and key to:

- Certificate: C:\nginx\conf\ssl\test-clientiq.crt
- Private Key: C:\nginx\conf\ssl\test-clientiq.key

- CA Bundle (if needed): C:\nginx\conf\ssl\ca-bundle.crt

Step 2: Create Nginx Configuration

Create C:\nginx\conf\nginx.conf :

```
worker_processes auto;
error_log logs/error.log warn;
pid logs/nginx.pid;

events {
    worker_connections 1024;
}

http {
    include mime.types;
    default_type application/octet-stream;

    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" rt=$request_time';

    access_log logs/access.log main;

    sendfile on;
    keepalive_timeout 65;
    gzip on;

    upstream clientiq_backend {
        server 127.0.0.1:5000;
        keepalive 32;
    }

    include conf.d/*.conf;
}
```

Step 3: Create Site Configuration

Create C:\nginx\conf\conf.d\test-clientiq.conf :

```

# =====
# ClientIQ Dev/Test Site Configuration
# Domain: test-clientiq.fmb.com
# =====

# HTTP - Redirect to HTTPS
server {
    listen 80;
    server_name test-clientiq.fmb.com;

    location /health {
        return 200 'OK';
        add_header Content-Type text/plain;
    }

    location / {
        return 301 https://$host$request_uri;
    }
}

# HTTPS - Main configuration
server {
    listen 443 ssl http2;
    server_name test-clientiq.fmb.com;

    # SSL Configuration
    ssl_certificate      ssl/test-clientiq.crt;
    ssl_certificate_key  ssl/test-clientiq.key;

    # SSL Security Settings
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-
    ssl_prefer_server_ciphers off;
    ssl_session_timeout 1d;
    ssl_session_cache shared:SSL:10m;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;

    # Client limits
    client_max_body_size 10M;

    # Logging
    access_log logs/test-clientiq_access.log main;
    error_log logs/test-clientiq_error.log warn;

    # Health check
    location /health {
        proxy_pass http://clientiq_backend/api/health;
        proxy_http_version 1.1;
        access_log off;
    }

    # API endpoints
    location /api/ {
        proxy_pass http://clientiq_backend;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Host $host;

        # WebSocket support
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}

```



```

        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }

    # SAML endpoints
    location /api/auth/saml/ {
        proxy_pass http://clientiq_backend;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_buffer_size 16k;
        proxy_buffers 4 16k;
        proxy_read_timeout 120s;
    }

    # Static assets
    location /assets/ {
        proxy_pass http://clientiq_backend;
        proxy_http_version 1.1;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Frontend (SPA)
    location / {
        proxy_pass http://clientiq_backend;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        add_header Cache-Control "no-cache, no-store, must-revalidate";
    }
}

```

Step 4: Install Nginx as Service

```

# Test configuration
cd C:\nginx
.\nginx.exe -t

# Install as service
C:\nssm\win64\nssm.exe install Nginx "C:\nginx\nginx.exe"
C:\nssm\win64\nssm.exe set Nginx AppDirectory "C:\nginx"
C:\nssm\win64\nssm.exe set Nginx DisplayName "Nginx HTTP Server"
C:\nssm\win64\nssm.exe set Nginx Description "Nginx reverse proxy for ClientIQ Dev/Test"
C:\nssm\win64\nssm.exe set Nginx Start SERVICE_AUTO_START

# Start the service
C:\nssm\win64\nssm.exe start Nginx

```

Firewall Configuration

```
# Allow HTTP (for redirect)
New-NetFirewallRule -DisplayName "HTTP (Dev/Test)" -Direction Inbound -LocalPort 80 -Protocol TCP -Action Allow

# Allow HTTPS
New-NetFirewallRule -DisplayName "HTTPS (Dev/Test)" -Direction Inbound -LocalPort 443 -Protocol TCP -Action Allow

# Allow SQL Server outbound
New-NetFirewallRule -DisplayName "SQL Server Outbound" -Direction Outbound -RemotePort 1433 -Protocol TCP -Action Allow
```

Verification Steps

1. Check Services

```
Get-Service -Name "ClientIQ-Dev", "Nginx" | Format-Table Name, Status, StartType
```

2. Test Application Health

```
# Direct test
Invoke-RestMethod -Uri "http://localhost:5000/api/health" -Method GET

# Via Nginx
Invoke-RestMethod -Uri "https://test-clientiq.fmb.com/api/health" -Method GET -SkipCertificateCheck
```

3. Check Logs

```
# Application logs
Get-Content "C:\ClientIQ\logs\stdout.log" -Tail 50

# Nginx logs
Get-Content "C:\nginx\logs\test-clientiq_access.log" -Tail 20
```

Service Management

```
# Start services
C:\nssm\win64\nssm.exe start ClientIQ-Dev
C:\nssm\win64\nssm.exe start Nginx

# Stop services
C:\nssm\win64\nssm.exe stop ClientIQ-Dev
C:\nssm\win64\nssm.exe stop Nginx

# Restart services
C:\nssm\win64\nssm.exe restart ClientIQ-Dev
C:\nssm\win64\nssm.exe restart Nginx

# View service status
C:\nssm\win64\nssm.exe status ClientIQ-Dev
C:\nssm\win64\nssm.exe status Nginx

# Edit service configuration (opens GUI)
C:\nssm\win64\nssm.exe edit ClientIQ-Dev
```

Troubleshooting

NODE_ENV Still Not Working

If you still see NODE_ENV errors after using the batch file or PowerShell script:

1. Verify environment variable is set in current session:

```
# PowerShell
$env:NODE_ENV

# Command Prompt
echo %NODE_ENV%
```

2. Try running tsx directly with env vars (PowerShell):

```
$env:NODE_ENV = "development"
npx tsx server/index.ts
```

3. Try running tsx directly with env vars (Command Prompt):

```
set NODE_ENV=development
npx tsx server/index.ts
```

4. Check if tsx is available:

```
npx tsx --version
```

ENOTSUP Error: Operation Not Supported on Socket

Symptoms:

```
Error: listen ENOTSUP: operation not supported on socket 0.0.0.0:5000
    at Server.setupListenHandle [as _listen2] (node:net:1918:21)
    ...
code: 'ENOTSUP',
errno: -4049,
syscall: 'listen',
address: '0.0.0.0',
port: 5000
```

Cause: Windows Server may not support binding to 0.0.0.0 on certain configurations.

Solution: Set HOST=127.0.0.1 instead of 0.0.0.0:

```
# PowerShell
$env:HOST = "127.0.0.1"

# Or in .env file
HOST=127.0.0.1

# Or in batch file
set HOST=127.0.0.1
```

Why this works: Since Nginx is handling external traffic and proxying to the Node.js app, the app only needs to listen on localhost (127.0.0.1). Nginx connects to http://127.0.0.1:5000 from the same server.

Application Won't Start

1. Check logs:

```
Get-Content "C:\ClientIQ\logs\stderr.log" -Tail 50
```

2. Verify dependencies:

```
cd C:\ClientIQ
npm install
```

3. Test manually:

```
$env:NODE_ENV = "development"
$env:DATABASE_DIALECT = "sqlserver"
# Set other required env vars...
npx tsx server/index.ts
```

Database Connection Failed

```
# Test SQL Server connectivity
Test-NetConnection -ComputerName "your-sql-server.fmb.com" -Port 1433

# Verify credentials using sqlcmd (if installed)
sqlcmd -S your-sql-server.fmb.com -U svc_clientiq_dev -P YourDevPassword -d ClientIQ_Dev -Q "SELECT 1"
```

Assumptions Made

1. **NSSM Path:** NSSM is installed at C:\nssm\win64\nssm.exe
2. **Node.js Path:** Node.js is installed at default location C:\Program Files\nodejs
3. **SQL Server Authentication:** Using SQL Server authentication (not Windows Authentication)
4. **SSL Certificate:** Certificate files are test-clientiq.crt and test-clientiq.key in C:\nginx\conf\ssl\
5. **SAML Disabled for Testing:** SAML authentication commented out; enable if using SSO in test environment
6. **Development Mode:** Running TypeScript source via tsx for easier debugging

7. **No External Package Downloads:** cross-env and similar packages cannot be downloaded; using batch/PowerShell scripts instead
-

Next Steps

1. [SSL Certificate Setup](#) - For proper certificate installation
 2. [SAML Configuration](#) - If using SSO in test environment
 3. [Troubleshooting](#) - For additional issues
-

Version History

Version	Date	Changes
1.0.0	December 2025	Initial dev/test server documentation